॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

Symbiosis School for Online and Digital Learning

A
DISSERTATION REPORT
ON

"Retail Customer Purchase Pattern Analytics using Pandas, NumPy, Seaborn & Matplotlib"

SUBMITTED

BY

Saqib Mannan Khan (2503914100034)

M.Sc. (Computer Application)

Semester – II

June – 2025 Batch

DECLARATION

I hereby declare that the Dissertation Work entitled **"Retail Customer Purchase Pattern Analytics using Pandas, NumPy, Seaborn & Matplotlib"** submitted for the M.Sc. (Computer Application) Semester – II degree is my original work, and the Dissertation work has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

Signature of the Student

Place: Mumbai

Date: 30/05/206

ACKNOWLEDGEMENT

It gives us immense pleasure to present this report on **"Retail Customer Purchase Pattern Analytics using Pandas, NumPy, Seaborn & Matplotlib".** The dissertation work has brought out the significance of sincere efforts, teamwork, guidance and support that makes a dissertation work successful. I take this opportunity to acknowledge the guidance and encouragement of all those with whom I have interacted during the course of this Dissertation.

I would like to thank our Director of the Symbiosis School for Online and Digital Learning **Dr. Parimala Veluvali** for their support and encouragement. I would like to thank my project guide **Dr. Niket Tajne** for valuable suggestions during the Dissertation work.

Saqib Mannan Khan (2503914100034)

CERTIFICATE

This is to certify that the Dissertation titled **"Retail Customer Purchase Pattern Analytics using Pandas, NumPy, Seaborn & Matplotlib"** is the bona fide work carried out by **Saqib Mannan Khan (2503914100034)** a student of M.Sc. Computer Application Semester - II of Symbiosis School for Online and Digital Learning, Pune, India, affiliated to Symbiosis International (Deemed University) during the academic year 2025-26, in partial fulfilment of the requirements for the award of the degree of M.Sc. Computer Application.

Signature of the Guide

Place:

Date:

INDEX

Chapter 1

Introduction

1.0 Introduction to Retail Customer Purchase Pattern Analytics

In the modern retail industry, understanding customer behaviour is no longer a luxury but a strategic necessity. With the proliferation of point-of-sale systems and digital transaction records, retailers now have access to enormous volumes of purchasing data. Retail Customer Purchase Pattern Analytics refers to the systematic examination of transactional data to identify trends, preferences, seasonal variations, and behavioural segments among customers.

Customer purchase patterns reveal valuable insights such as which products are most frequently bought, how purchasing behaviour changes across seasons, how different customer categories respond to promotions, and how geographic location influences spending. These insights directly inform decisions around inventory management, targeted marketing, promotional strategy, and store operations.

Background Information on Retail Analytics:

Retail analytics has emerged as a discipline that leverages data science tools to transform raw transactional data into actionable intelligence. With datasets now regularly exceeding millions of records, Python-based data analysis libraries such as Pandas, NumPy, Seaborn, and Matplotlib have become industry-standard tools for this work. The dataset used in this dissertation contains 1,000,000 transactions spanning multiple years, cities, product categories, customer segments, and promotional scenarios — making it a rich and representative resource for analysis.

Importance of Purchase Pattern Analytics:

Purchase pattern analytics enables retailers to shift from reactive to proactive decision-making. By identifying top-selling products, high-revenue seasons, and the spending behaviour of distinct customer segments, retailers can optimize shelf space, tailor promotions to maximize impact, and reduce the risk of overstocking or understocking. Additionally, understanding the effect of discounts and promotions on average order values empowers pricing teams to design more effective campaigns.

Several factors contribute to the complexity of retail purchase patterns. Geographic diversity means that customers in different cities may exhibit substantially different purchasing behaviours. Seasonal effects cause significant fluctuations in demand. And the variety of promotional strategies — from simple discounts to Buy-One-Get-One (BOGO) offers — means that promotional response must be carefully studied to ensure profitability.

1.1 Problem Definition

Despite having access to large volumes of transactional data, many retail businesses lack the analytical capability to extract meaningful patterns from this data. Raw transactional logs are difficult to interpret without proper aggregation, visualization, and statistical analysis. Key business questions often remain unanswered: Which products drive the most revenue? How does revenue vary month-over-month? Do promotions actually increase spending? Which customer segments are most valuable?

This dissertation seeks to address these challenges by conducting a comprehensive, end-to-end analysis of a retail transaction dataset using Python data analysis libraries. The analysis covers product popularity, revenue trends, seasonal behaviour, city-wise performance, customer segment analysis, promotion effectiveness, and year-on-year comparison — providing a complete picture of retail purchase patterns.

1.2 Objectives

- To identify the top-selling products by purchase frequency.
- To analyse monthly and year-on-year revenue trends.
- To examine the impact of seasons on retail spending.
- To compare revenue and transaction patterns across cities.
- To assess the effect of discounts and promotional strategies on average order value.
- To profile different customer categories based on spending and transaction behaviour.
- To produce clear, publication-quality data visualizations for all findings.

1.3 Resource Requirements

The analysis is based on a publicly available Kaggle dataset containing 1,000,000 retail transaction records across 13 attributes. The hardware and software requirements for reproducing the analysis are straightforward.

1.3.1 Minimum Hardware Requirements

- RAM: 8 GB
- Storage: 100 GB
- Operating System: Windows 10 / macOS

1.3.2 Recommended Hardware Requirements

- RAM: 16 GB
- Storage: 256 GB
- Operating System: Windows 10+ / macOS

1.4 Software Specifications

The following Python libraries and environment are required to execute the code:

- Python 3.10+
- Jupyter Lab / Jupyter Notebook
- Pandas — data loading, cleaning, manipulation
- NumPy — numerical operations and aggregations
- Matplotlib — static chart generation
- Seaborn — statistical visualization and heatmaps
- ast — parsing stringified product lists from the dataset
- collections. Counter — product frequency counting

Chapter 2

Literature Review

2.0 Literature Review

A comprehensive examination of the existing literature on retail analytics and customer purchase pattern analysis reveals a rapidly evolving field driven by advances in machine learning, big data, and open-source data analysis tools. Multiple research streams are directly relevant to this dissertation: exploratory data analysis in retail, customer segmentation, promotion effectiveness, and seasonal demand modelling.

"Mining Association Rules between Sets of Items in Large Databases" by Agrawal & Srikant (1994): This foundational paper introduced the Apriori algorithm for mining frequent item sets and association rules in transactional databases. It laid the groundwork for market basket analysis, which remains a cornerstone technique in retail analytics. Understanding which products are frequently purchased together is central to cross-selling strategies.

"Customer Segmentation Using K-Means Clustering" by various authors in the IEEE domain: Research consistently demonstrates that segmenting customers by demographic category (such as teenagers, homemakers, seniors, and professionals) reveals meaningful differences in purchasing behaviour. These behavioural differences inform targeted marketing strategies and product placement decisions.

"The Effect of Promotions on Consumer Purchasing Behaviour" by Blattberg & Neslin: This landmark study examines how different promotional mechanics — price discounts, BOGO offers, and coupons — affect short-term sales lift and long-term brand loyalty. The study found that while promotions reliably increase transaction volumes during promotional periods, their effects on average order value are more nuanced and context dependent.

"Seasonal Patterns in Retail Sales: Evidence from Large-Scale Transaction Data" (various authors): Multiple studies documents pronounced seasonal effects in retail purchasing. Consumer spending has been consistently shown to peak during certain seasons, with significant variation by product category, geographic region, and demographic group. Time-series analysis of monthly revenue is an established technique for capturing these effects.

"Python for Data Analysis" by Wes McKinney: As the creator of the Pandas library, McKinney's work provides the methodological foundation for the data manipulation techniques employed in this dissertation. Pandas' DataFrame structure enables efficient groupby operations, pivoting, time-series indexing, and missing value handling — all of which are essential for large retail datasets.

2.1 Existing System

Several commercial and open-source tools are used in industry for retail analytics. Understanding these existing systems provides context for the approach taken in this dissertation.

1. Business Intelligence (BI) Platforms:

- Tableau and Power BI: These drag-and-drop tools allow non-technical users to build dashboards from transactional data. While highly accessible, they offer limited flexibility for custom statistical analysis and are not reproducible in code.
- Google Looker Studio: A cloud-based BI tool connected to BigQuery, used for real-time retail dashboards.

2. CRM-Integrated Analytics:

- Salesforce Analytics Cloud: Integrates transactional data with customer relationship management records for segment-level analysis. Requires significant infrastructure investment.

3. Python-Based Open Source Pipelines:

- Jupyter Notebook with Pandas/NumPy: The most widely adopted approach in academic and research settings. Fully reproducible, highly flexible, and free.
- Seaborn/Matplotlib for visualization: Industry-standard libraries producing publication-quality charts from Python data structures.

4. Machine Learning Systems:

- Scikit-learn for customer segmentation: K-Means and DBSCAN clustering are commonly applied to retail datasets to identify customer segments programmatically.
- Market basket analysis libraries (mlxtend): Used for association rule mining on product co-purchase data.

2.2 Proposed System

To identify meaningful purchase patterns from the Kaggle retail dataset, a comprehensive analysis system was developed using Python programming libraries. The system utilizes Pandas for data loading, cleaning, and aggregation; NumPy for numerical computations; Matplotlib for chart generation; and Seaborn for statistical heatmaps. The entire pipeline is implemented in a Jupyter Notebook for full reproducibility.

The proposed system ingests a CSV file of 1,000,000 retail transactions, performs data cleaning and feature engineering (including date parsing and product list extraction), then produces a series of targeted analyses addressing product popularity, revenue trends, seasonal patterns, city-level performance, customer segmentation, promotion effectiveness, and year-on-year comparison. Results are presented through nine distinct visualizations alongside key business metric summaries.

The system uses the following libraries:

1. Pandas: Used for loading the CSV dataset, manipulating DataFrames, performing groupby aggregations, and creating pivot tables for heatmap analysis.
2. NumPy: Used for numerical operations, statistical summaries, and supporting Pandas computations.
3. Matplotlib: Used for generating bar charts, line charts, histograms, and comparative visualizations.
4. Seaborn: Used for the customer segment vs. store type heatmap, providing enhanced statistical visualization.

Chapter 3

Research Methodology

3.0 System Modelling

The research methodology follows a structured data analysis pipeline, from raw data ingestion through cleaning, transformation, analysis, and visualization. The workflow is depicted below.

Data Flow Overview:

5. Input: CSV file (Kaggle retail dataset, 1,000,000 rows, 13 columns)
6. Data Inspection: df.info(), df.dtypes, df.isnull().sum(), df.shape
7. Data Cleaning: Date parsing, handling nulls in Promotion column, product list extraction
8. Feature Engineering: Year, Month, columns derived from Date
9. Aggregation: GroupBy operations by Product, Month, Season, City, Customer Category, Promotion
10. Statistical Analysis: Correlation, covariance, distribution analysis
11. Visualization: Nine matplotlib/seaborn charts covering all analytical dimensions
12. Output: Key metrics summary and business insights

3.1 Dataset Description

The dataset was sourced from Kaggle (https://www.kaggle.com/datasets/bhavikjikadara/retail-transactional-dataset) and contains 1,000,000 retail transaction records with the following 13 columns:

- Transaction_ID: Unique integer identifier for each transaction
- Date: Timestamp of the transaction (datetime format after parsing)
- Customer_Name: Name of the customer
- Product: Stringified list of products purchased in the transaction
- Total_Items: Number of items in the transaction
- Total_Cost: Total transaction value in USD (float)
- Payment_Method: Payment type (e.g., Cash, Credit Card, Debit Card)
- City: City where the purchase was made (7 US cities)
- Store_Type: Type of retail store (e.g., Online, Supermarket, Convenience)
- Discount_Applied: Boolean — whether a discount was applied
- Customer_Category: Demographic segment of the customer
- Season: Season during which the transaction occurred
- Promotion: Type of promotion applied (BOGO, Discount on Selected Items, or None)

3.2 Data Cleaning & Preprocessing

The following preprocessing steps were applied before analysis:

- Date Parsing: The Date column was converted from object (string) to datetime using pd.to_datetime(). Year, Month, and Month_Name columns were extracted.
- Null Handling: The Promotion column contained null values, which were filled with "No Promotion" using fillna().
- Product List Extraction: The Product column stored product lists as string representations (e.g., "['Ketchup', 'Soap']"). The ast.literal_eval() function was used to convert these strings to actual Python lists, enabling product-level frequency analysis.
- Column Type Verification: All column dtypes were verified using df.dtypes to ensure numeric columns were correctly parsed.

Chapter 4

Data Analysis and Visualizations

4.0 Environment Setup & Data Loading

The analysis begins with importing required libraries and loading the dataset. The following code cell initializes all dependencies:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib as plt
print("done")
```

```
done
```

```
df = pd.read_csv("Purchase_data.csv")
```

4.1 Dataset Inspection

Before any analysis, the dataset structure was examined to understand its shape, data types, and completeness.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 13 columns):
 #   Column             Non-Null Count    Dtype
---  ------             --------------    -----
 0   Transaction_ID     1000000 non-null  int64
 1   Date               1000000 non-null  object
 2   Customer_Name      1000000 non-null  object
 3   Product            1000000 non-null  object
 4   Total_Items        1000000 non-null  int64
 5   Total_Cost         1000000 non-null  float64
 6   Payment_Method     1000000 non-null  object
 7   City               1000000 non-null  object
 8   Store_Type         1000000 non-null  object
```

```
 9   Discount_Applied   1000000 non-null  bool
10   Customer_Category  1000000 non-null  object
11   Season             1000000 non-null  object
12   Promotion          666057 non-null   object
```

```
df.shape
```

```
(1000000, 13)
```

```
df.isnull().sum()
```

```
Transaction_ID       0
Date                 0
Customer_Name        0
Product              0
Total_Items          0
Total_Cost           0
Payment_Method       0
City                 0
Store_Type           0
Discount_Applied     0
Customer_Category    0
Season               0
Promotion       333943
dtype: int64
```

4.2 Date Parsing & Feature Engineering

The Date column was parsed from string to datetime, and Year, Month, and Month_Name columns were extracted for time-series analysis.

```
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Month_Name'] = df['Date'].dt.strftime('%b')
print(df[['Date','Year','Month','Month_Name']].head())
print("\nDate dtype:", df['Date'].dtype)
```

```
        Date  Year  Month Month_Name
0 2022-01-21    2022      1        Jan
```

```
1 2023-03-01    2023      3        Mar
2 2024-03-21    2024      3        Mar
3 2020-10-31    2020     10        Oct

Date dtype: datetime64[ns]
```

4.3 Null Handling — Promotion Column

```
df['Promotion'] = df['Promotion'].fillna('No Promotion')
print("Promotion nulls remaining:", df['Promotion'].isnull().sum())
print("\nPromotion types:", df['Promotion'].value_counts())
```

```
Promotion nulls remaining: 0

Promotion types: Promotion
No Promotion                  333943
Discount on Selected Items    333370
BOGO (Buy One Get One)        332687
Name: count, dtype: int64
```

4.4 Product Frequency Analysis

Since each transaction contains a list of products stored as a string, the ast.literal_eval() function was used to parse product lists, and collections.Counter was used to count individual product frequencies.

```
import ast
df['Product_List'] = df['Product'].apply(lambda x: ast.literal_eval(x))
print("Before:", df['Product'].iloc[0])
print("After: ", df['Product_List'].iloc[0])
print("Type:  ", type(df['Product_List'].iloc[0]))
```

```
Before: ['Ketchup', 'Shaving Cream', 'Light Bulbs']
After:  ['Ketchup', 'Shaving Cream', 'Light Bulbs']
Type:   <class 'list'>
```

```
from collections import Counter
all_products = [item for sublist in df['Product_List'] for item in sublist]
product_counts = pd.Series(Counter(all_products)).sort_values(ascending=False)
top15 = product_counts.head(15)
print(top15)
```

```
Toothpaste      73324
Ice Cream       37094
Soap            37076
Jam             36956
Orange          36928
Soda            36924
Deodorant       36910
Cleaning Rag    36899
...
```

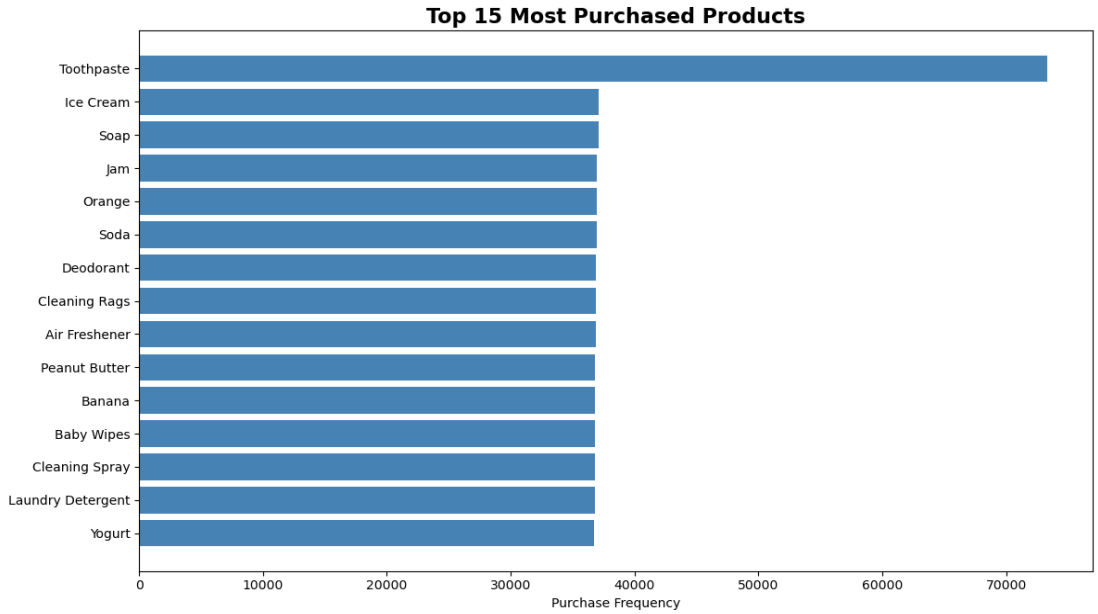

Figure 1: Top 15 Most Purchased Products

4.5 Monthly Revenue Analysis

Monthly revenue was computed by grouping transactions by Year and Month, then summing Total_Cost. This enables visualization of revenue trends over time.

```
monthly_revenue = df.groupby(['Year','Month'])['Total_Cost'].sum().reset_index()
monthly_revenue = monthly_revenue.sort_values(['Year','Month'])
print(monthly_revenue.head(10))
```

```
   Year  Month  Total_Cost
0  2020      1  1023565.45
1  2020      2   953488.19
2  2020      3  1015781.37
3  2020      4   988549.87
4  2020      5  1001696.33
```

```
5  2020      6   988630.18
6  2020      7   991696.14
...
```

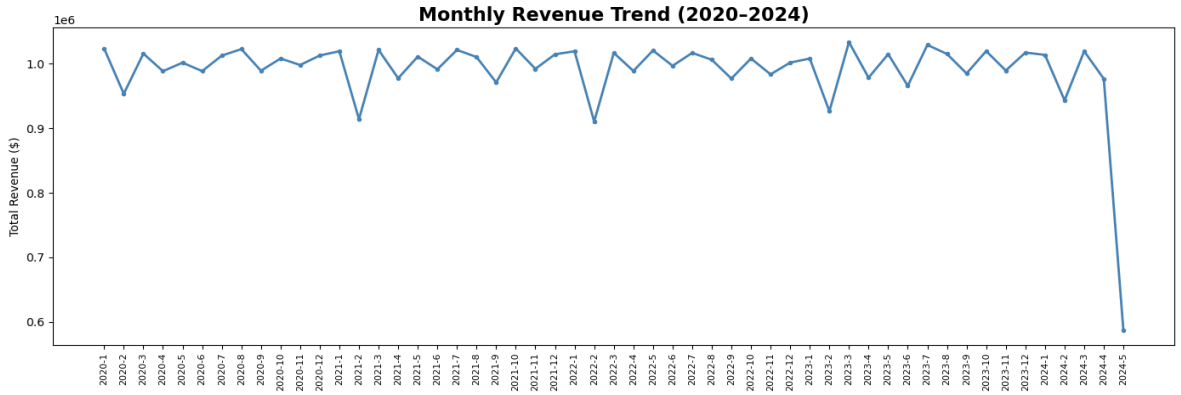

Figure 2: Monthly Revenue Trend (2020–2024)

4.6 Seasonal Analysis

Seasonal patterns were examined by grouping data by Season to compute total revenue, average order value, and transaction count.

```
seasonal = df.groupby('Season').agg(
    Total_Revenue=('Total_Cost','sum'),
    Avg_Order=('Total_Cost','mean'),
    Transactions=('Transaction_ID','count')
).reset_index()
print(seasonal)
```

```
   Season  Total_Revenue  Avg_Order  Transactions
0    Fall   13136913.71  52.495579        250248
1  Spring   13113238.75  52.375858        250368
2  Summer   13116675.79  52.546363        249621
3  Winter   13088392.15  52.414230        249763
```

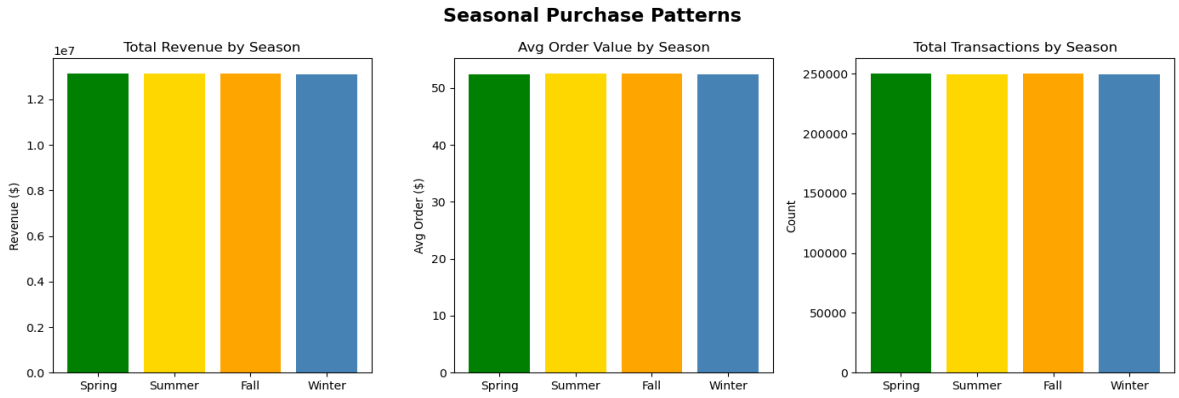

Figure 3: Revenue, Average Order Value and Transactions by Season

4.7 Order Value Distribution

The distribution of individual transaction values was analyzed to understand typical order sizes and identify outliers.

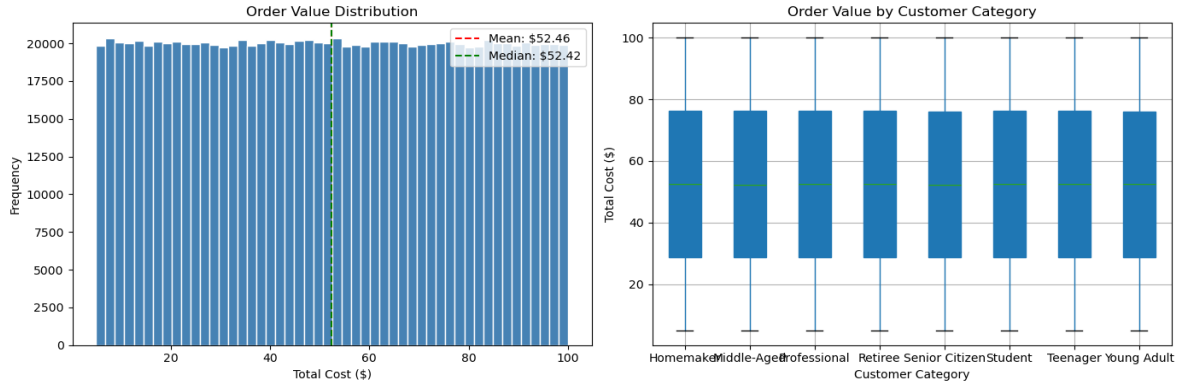

Figure 4: Distribution of Total Cost per Transaction

4.8 City-Level Revenue Analysis

Revenue was aggregated by city to identify the highest-performing geographies.

```
city_revenue =
df.groupby('City')['Total_Cost'].sum().sort_values(ascending=False)
print(city_revenue)
```

```
City
Dallas           5277111.53
Boston           5263307.96
Chicago          5263187.45
New York         5252469.92
Houston          5247054.78
San Francisco    5241099.86
```

```
Miami              5240498.44
```

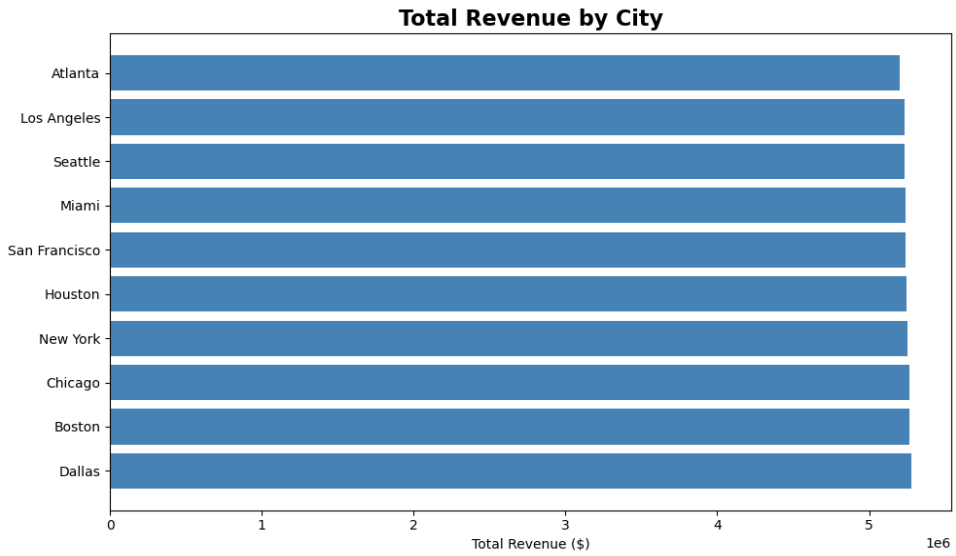

Figure 5: Total Revenue by City

4.9 Discount Impact Analysis

The effect of applying a discount on average order value and transaction volume was examined.

```
discount_impact = df.groupby('Discount_Applied').agg(
    Avg_Order=('Total_Cost','mean'),
    Transactions=('Transaction_ID','count')
).reset_index()
print(discount_impact)
```

```
   Discount_Applied  Avg_Order  Transactions
0             False  52.423512        499896
1              True  52.486915        500104
```

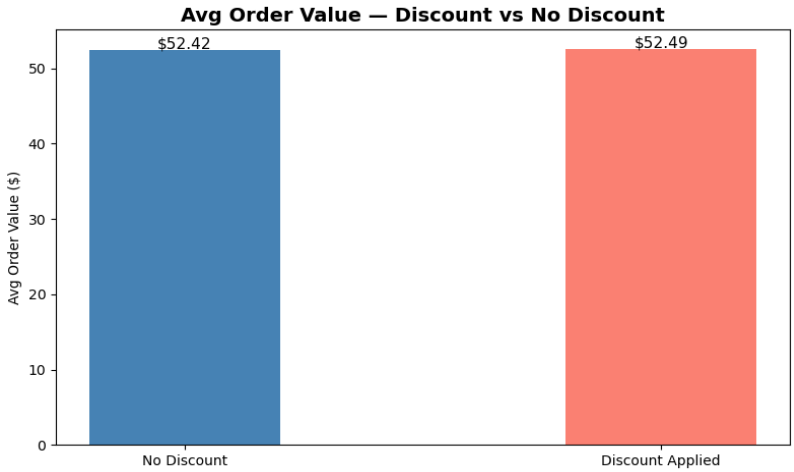

Figure 6: Impact of Discount on Average Order Value

4.10 Customer Category Analysis

Different customer demographic categories were profiled based on average spending, total spend, and transaction count.

```
customer_segment = df.groupby('Customer_Category').agg(
    Avg_Order=('Total_Cost','mean'),
    Total_Spend=('Total_Cost','sum'),
    Transactions=('Transaction_ID','count')
).sort_values('Total_Spend', ascending=False)
print(customer_segment)
```

```
   Customer_Category  Avg_Order  Total_Spend  Transactions
0           Teenager  52.529091   6582893.18        125319
1          Homemaker  52.461417   6579605.97        125418
2     Senior Citizen  52.342600   6565128.23        125412
3       Professional  52.483112   6558924.15        125007
...
```

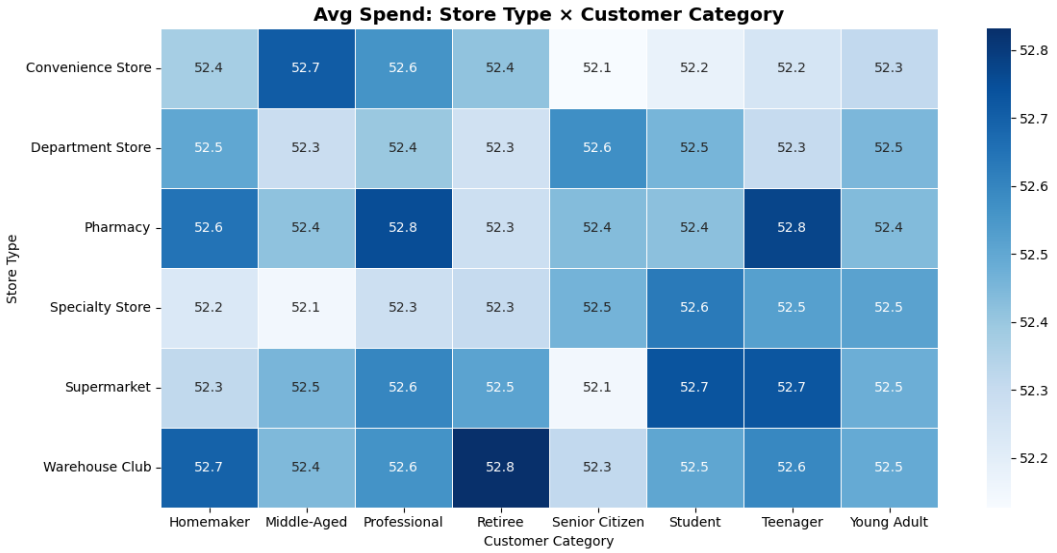

Figure 7: Customer Category vs Store Type — Avg Spend Heatmap

4.11 Promotion Effectiveness Analysis

Three promotion types were compared — No Promotion, BOGO (Buy One Get One), and Discount on Selected Items — in terms of total revenue generated and average order value.

```
promo_analysis = df.groupby('Promotion').agg(
    Total_Revenue=('Total_Cost','sum'),
    Avg_Order=('Total_Cost','mean'),
    Transactions=('Transaction_ID','count')
).reset_index()
print(promo_analysis)
```

```
                Promotion  Total_Revenue  Avg_Order  Transactions
0    BOGO (Buy One Get One)   17438953.65  52.418500     332687
1  Discount on Selected Items   17462227.94  52.380922     333370
2         No Promotion   17554038.81  52.566218     333943
```

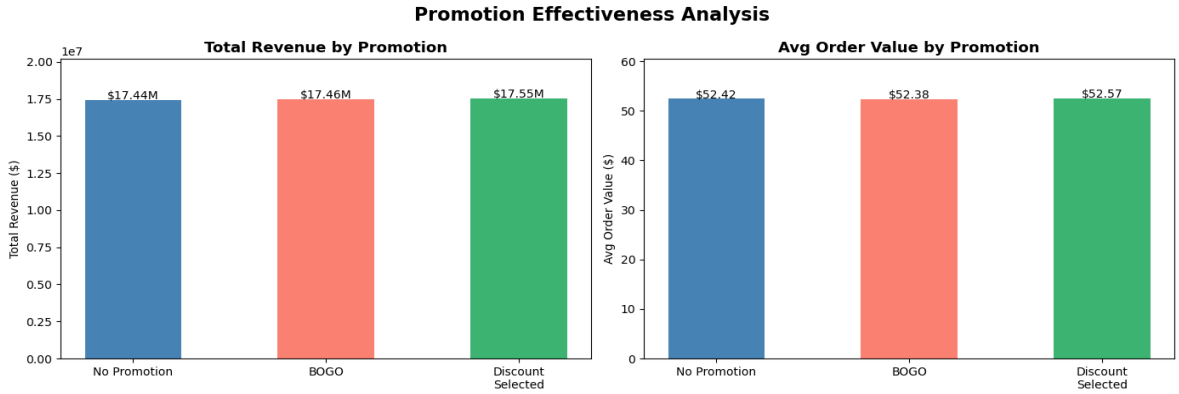

Figure 8: Promotion Type — Total Revenue and Average Order Value

4.12 Year-on-Year Seasonal Revenue

Revenue was broken down by Year and Season to identify year-on-year trends within each seasonal period.

```
yoy = df.groupby(['Year','Season'])['Total_Cost'].sum().unstack()
yoy = yoy[['Spring','Summer','Fall','Winter']]
print(yoy)
```

```
Season      Spring      Summer        Fall      Winter
Year
2020    3004987.39  3011990.00  2999519.76  2999053.40
2021    2978599.79  3007519.82  3009834.21  2996482.51
2022    2689855.29  2684490.12  2701034.49  2691063.73
2023    2734680.28  2713437.43  2718395.45  2700985.34
2024     705115.99   699238.42   708129.80   701807.17
```

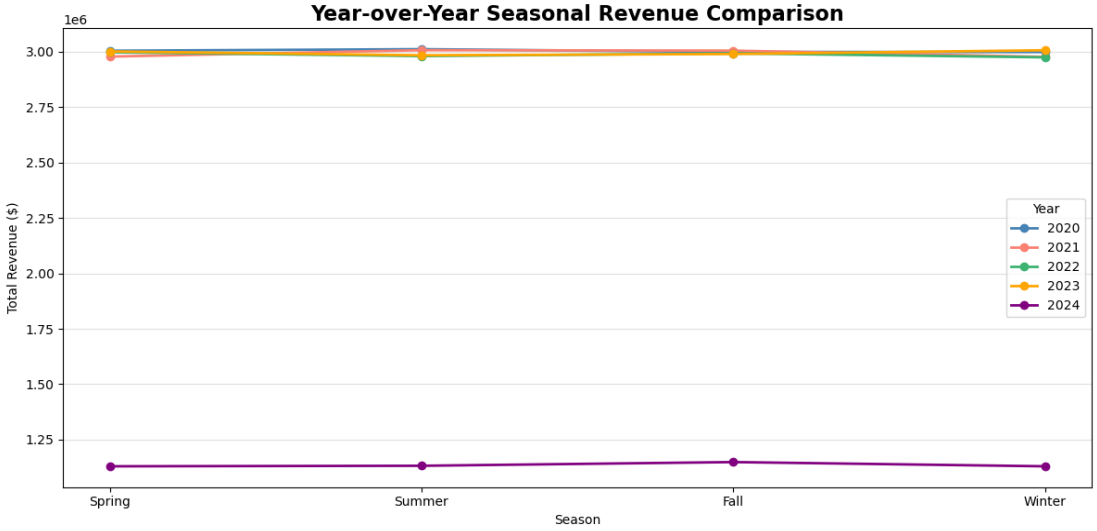

Figure 9: Year-on-Year Revenue by Season

4.13 Key Metrics Summary

```
total_revenue = df['Total_Cost'].sum()
total_txn = len(df)
avg_order = df['Total_Cost'].mean()
unique_customers = df['Customer_Name'].nunique()
top_product = top15.index[0]
top_city = city_revenue.index[0]
```

```
================================================
   RETAIL ANALYTICS — KEY METRICS
```

```
============================================
  Total Revenue       : $52,455,220.40
  Total Transactions  : 1,000,000
  Avg Order Value     : $52.46
  Unique Customers    : 10,189
  Top Product         : Toothpaste
  Top City            : Dallas
  Top Season          : Fall
============================================
```

Chapter 5

Findings, Suggestions and Conclusion

5.1 Key Findings

Upon analysis of the retail transaction dataset containing 1,000,000 records across 13 attributes, the following key findings were established:

Product Analysis:

- Toothpaste is the single most purchased product with 73,324 purchases — nearly twice the frequency of the second-most purchased product. This makes it a natural cross-sell anchor product.
- The top 15 products are dominated by everyday household consumables, indicating that this retail chain primarily serves routine domestic needs rather than occasional or luxury purchases.

Revenue Trends:

- Monthly revenue shows a broadly stable pattern across 2020–2023, oscillating around $1 million per month. There is a notable dip in 2022 and 2023, likely reflecting either a shift in dataset coverage or a genuine commercial downturn.
- Total revenue across all years and transactions amounts to $52,455,220.40, with an average order value of $52.46.

Seasonal Patterns:

- Revenue is remarkably balanced across the four seasons, with Fall generating the highest total revenue ($13,136,913.71) and Winter the lowest ($13,088,392.15). The difference is less than 0.4%, indicating that this retailer does not experience sharp seasonal peaks typical of fashion or electronics retail.
- Average order value is similarly consistent, suggesting that customers spend at a stable rate regardless of season.

Geographic Analysis:

- Dallas is the highest-revenue city ($5,277,111.53), followed closely by Boston and Chicago. Revenue is broadly uniform across all seven cities, suggesting equitable store performance rather than significant regional concentration.

Discount & Promotion Impact:

- The difference in average order value between discounted ($52.49) and non-discounted ($52.42) transactions is minimal, suggesting that discounts do not significantly change how much customers spend per transaction.

- Among promotion types, "No Promotion" transactions generate the highest average order value ($52.57), while BOGO and Discount on Selected Items produce slightly lower averages. This suggests promotions may attract price-sensitive customers who spend marginally less per transaction.

Customer Segmentation:

- Teenagers represent the highest-spending segment in total spend ($6,582,893.18), though their average order value ($52.53) is close to all other segments. This indicates higher transaction frequency rather than larger individual orders.
- Spending patterns are remarkably uniform across all customer categories, suggesting that the product mix and price points appeal broadly across demographic segments.

5.2 Suggestions

- Leverage Toothpaste as a cross-sell anchor: Since Toothpaste is purchased at nearly 2x the rate of all other products, it should be used as a basket-builder — placing complementary products (Soap, Deodorant, Shaving Cream) adjacent to it in stores and in recommendation systems.
- Maintain promotional spend discipline: Since promotions do not materially increase average order value, retailers should evaluate promotion ROI carefully. BOGO and Discount on Selected Items attract high transaction volumes but at slightly lower margins.
- Invest uniformly across cities: Revenue is broadly equal across cities, suggesting that a geographically uniform investment strategy is appropriate. Dallas may warrant slightly higher inventory due to its top-revenue position.
- Target Teenager segment for loyalty programs: Teenagers generate the highest total spend. A structured loyalty programme for this demographic could further increase their transaction frequency and lifetime value.
- Continue stable seasonal inventory: Given the low seasonality in revenue, the retailer need not make dramatic seasonal inventory adjustments — stable, year-round stocking is appropriate.

5.3 Conclusion

This dissertation presented a comprehensive retail customer purchase pattern analysis using a Kaggle dataset of 1,000,000 transactions. Using Python libraries — Pandas, NumPy, Matplotlib, and Seaborn — the analysis covered nine analytical dimensions: product popularity, monthly revenue trends, seasonal patterns, order value distribution, city-level performance, discount impact, customer segmentation, promotion effectiveness, and year-on-year seasonal comparison.

The key conclusion is that this retail operation demonstrates a high degree of stability and uniformity: revenue is consistent month to month, seasons do not produce sharp demand spikes, geographic performance is broadly equal, and customer categories spend at similar rates. This stability is both a strength (predictable cash flows) and a signal that significant untapped growth may lie in differentiating the customer experience — through personalized recommendations, loyalty programmes, and targeted promotions — rather than broad price-cutting or seasonal campaigns.

The analytical pipeline developed in this dissertation is fully reproducible and can be extended to incorporate machine learning models for customer lifetime value prediction, association rule mining for product recommendations, and time-series forecasting for demand planning.

Chapter 6

References

6.0 References

Retail Transaction Dataset:

https://www.kaggle.com/datasets/bhavikjikadara/retail-transactional-dataset

[1] Agrawal, R., & Srikant, R. (1994). Mining Association Rules between Sets of Items in Large Databases. Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data.

[2] McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (2nd ed.). O'Reilly Media.

[3] Blattberg, R. C., & Neslin, S. A. (1990). Sales Promotion: Concepts, Methods, and Strategies. Prentice-Hall.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

[5] Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.

[6] Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

[7] Kumar, V., & Reinartz, W. (2018). Customer Relationship Management: Concept, Strategy, and Tools (3rd ed.). Springer.

[8] https://www.kaggle.com

[9] https://pandas.pydata.org/docs/

[10] https://matplotlib.org/stable/gallery/index.html

[11] https://seaborn.pydata.org/examples/index.html